harwood — decisões de projeto

Convenções fixadas antes de retomar o desenvolvimento

Este documento registra o que foi decidido nas conversas de projeto do **harwood**, o substituto do IRAF em Python. Ele é o contrato de design: quando uma dúvida de implementação aparecer, a resposta deveria estar aqui.

Companheiro deste documento: *IRAF — inventário de funcionalidades e o que o torna referência*, que levanta as 609 tarefas do IRAF community v2.18.2, os seus pontos fortes e os seus tropeços. As decisões abaixo derivam dele.

Estado de cada item. *Decidido* é convenção fixada. *Proposto* é recomendação técnica ainda não confirmada. *Em aberto* é o que falta decidir, reunido na seção 10.

Sumário

1. O que é o harwood 1
2. Corrigir os tropeços do IRAF 2
3. Nomes, parâmetros e migração 2
4. Distribuição e instalação 4
5. Editor de parâmetros 4
6. Idioma 4
7. Onde o Python não basta 6
8. Visualizador nativo 7
9. Estado atual do código 9
10. Em aberto 9

1. O que é o harwood

Item	Decisão
Objetivo	Substituir o IRAF por um software mais amigável, sem perder funcionalidade nem capacidade
Motivação	O IRAF é complexo, datado e tem curva de aprendizado íngreme, mas não tem alternativa equivalente — daí a sensação de dependência
Base	Python é o esqueleto; outra linguagem entra só onde o Python não dá conta
Princípio central	Orquestração, não reimplementação: ccdproc, photutils e astroscrappy já têm os algoritmos; falta a casca de orquestração, validação e proveniência
Nome	Margaret Harwood (1885–1979), fotometrista, primeira mulher a dirigir um observatório independente. Também acrônimo: *Homogeneous Astronomical Reduction Workflows: Open, Observable, Documented*
Plataforma	Linux (Ubuntu como referência). Windows via WSL2, sem suporte oficial
Rede	Offline por padrão: sem navegador, sem servidor, sem dependência de internet
Horizonte	Projeto pessoal, sem prazo, com a intenção de um dia ser distribuído à comunidade astronômica

O objetivo de longo prazo, distribuir para a comunidade, é o que sustenta as decisões das seções 2 e 3: compatibilidade de nomes e instalação simples só importam quando existem usuários além de quem escreveu o código.

2. Corrigir os tropeços do IRAF

Cada linha vem da seção "Onde o IRAF tropeça" do inventário.

Tropeço do IRAF	Correção no harwood	Estado
Curva de aprendizado	Nomes do IRAF preservados; o que muda é a descoberta: scripts em Python comum, teclas exibidas na tela, busca por descrição (`harwood references`)	DECIDIDO
Estado oculto em `uparm$`	Nenhum valor aprendido em silêncio. Parâmetros ficam no diretório da redução; cada execução registra o valor efetivo e sua origem (padrão, projeto ou linha de comando)	DECIDIDO
Legado de hardware	Não portar fitas, IIS, Tektronix, plotters nem formatos históricos de observatório. FITS com MEF é o formato de trabalho	DECIDIDO
Incertezas não propagadas	Dados, variância e máscara andam juntos em toda operação, gravados como extensões `SCI`/`VAR`/`DQ` (convenção do DRAGONS)	*PROPOSTO*
WCS próprio	Ler TNX, ZPX e multispec na entrada, para migração; escrever sempre em convenções padrão (SIP, TPV, WCS espectral FITS)	*PROPOSTO*
Desempenho serial	NumPy vetorizado, paralelismo por arquivo, núcleos compilados onde necessário (seção 7)	DECIDIDO
Extensão via SPP	Uma tarefa é uma função Python mais um modelo Pydantic; pacotes de terceiros entram como plugins instaláveis	DECIDIDO
Rede	Offline por padrão; consulta a catálogos só como extra opcional	DECIDIDO
Manutenção	Poucas dependências obrigatórias; testes com dados sintéticos e regressão contra o IRAF nos mesmos dados	DECIDIDO
Instalação e display	Um comando para instalar; visualizador embutido, com DS9 opcional (seções 4 e 8)	DECIDIDO

2.1 Os "bastidores" visíveis

O ponto levantado em aula: em muitos softwares de redução, processos acontecem por trás dos panos e o usuário não sabe o que foi feito. O harwood ataca isso em três frentes.

- **Modo plano.** Antes de executar, toda tarefa mostra as etapas em ordem, com arquivos e valores reais. Exemplo: *overscan Legendre ordem 1 em [2049:2080,*] → trim → subtrair Zero.fits (10 frames combinados)*. O `noproc` do `ccdproc` é o embrião dessa ideia. **Decidido.**
- **Relatório pós-execução.** Estatísticas antes e depois de cada etapa. **Proposto.**
- **Editor de parâmetros que explica** (seção 5). **Decidido.**

2.2 Convenção de pixel

IRAF, FITS e DS9 contam pixels a partir de 1 e usam a ordem (x, y). NumPy e photutils contam a partir de 0 e indexam `[y, x]`.

Decisão: a interface do harwood é 1-based e (x, y), em toda a superfície visível — linha de comando, visualizador, relatórios, arquivos de coordenadas. A conversão acontece na fronteira com as bibliotecas. Expor a convenção do Python faria todo usuário vindo do IRAF errar coordenadas por um pixel.

3. Nomes, parâmetros e migração

Regra geral: mesmo nome, mesmo significado. Quem digita `imcombine --reject ccdclip` recebe o comportamento do IRAF ou um erro claro. Nome diferente atrapalha pouco; nome igual que faz outra coisa em silêncio estraga resultado científico.

1. **Tarefas** mantêm o nome do IRAF sempre que a tarefa existir lá.
2. **Parâmetros** mantêm nome e valores permitidos quando o significado for igual (`reject`, `lsigma`, `rdnoise`, `avsigclip`, `spline3`…).
3. **Memória muscular** também é preservada, e pesa mais que os nomes: teclas do `imexamine` e do `splot`, teclas do ajuste interativo (`f`, `d`, `u`, `:order`), sintaxe `img.fits[SCI,1][100:400,*]` e listas `@arquivo`.
4. **Divergir só em três casos:**
5. conflito com o Python — `dispcor` tem parâmetro `global`, palavra reservada: vira `global_` na API e continua `global` na linha de comando;

6. significado diferente — aí o nome precisa mudar;
7. conceito que só existe por legado (`imtype`, `stdimage`) — não é portado.
8. **Migração:** ler arquivos `.par` e a saída do `dpar`; ler o diretório `database/` do `identify` e do `apall`, para aproveitar soluções de dispersão já feitas. A documentação traz uma tabela de equivalência IRAF → harwood com as diferenças explícitas.

3.1 Sintaxe da linha de comando

Decidido: estilo POSIX, por modernização. Os nomes continuam os do IRAF; muda a forma de escrever.

```
harwood imcombine @bias.lis Zero.fits --combine median --reject ccdclip --no-interactive
```

Aspecto	Convenção	Estado
Parâmetros automáticos do IRAF (`input`, `output`)	Argumentos posicionais	DECIDIDO
Parâmetros ocultos	Opções `--nome valor`	DECIDIDO
Nomes compostos	`--low-reject` na linha de comando; `low_reject` continua canônico em Python, arquivos de parâmetros e receitas; aceitar também `--low_reject`	*PROPOSTO*
Booleanos	Par `--interactive` / `--no-interactive`	*PROPOSTO*
Opções curtas	Poucas e só globais (`-h`, `-v`, `-q`); letras por tarefa colidem e ninguém memoriza	*PROPOSTO*
Seções de imagem	Sempre entre aspas na documentação e no autocompletar: `'obj.fits[SCI,1][100:400,*]'`, porque colchetes e `*` são curingas do shell	DECIDIDO

3.2 Tabela de tarefas e `--help`

Quatro comandos, com papéis distintos — herança direta do `?`, `??`, `help` e `lpar` da CL do IRAF:

Comando	Papel
`list`	Navegar por pacotes e tarefas
`--help`	Referência rápida: descrição curta e parâmetros com tipo, padrão e valores permitidos, agrupados por tema
`help`	Manual completo: algoritmo, exemplos, referências
`lpar`	Valores que serão usados de fato e a origem de cada um

O erro a evitar é misturar os papéis: um `--help` que despeja o manual inteiro, ou que mostra o padrão como se fosse o valor em uso.

```
$ harwood list ccdred
Tarefa         Status        Descrição
ccdproc        disponível    Processa imagens CCD
zerocombine    planejada     Combina e processa imagens de bias
setinstrument  substituída   → arquivo de instrumento
```

Convenções de apoio:

- **Fonte única:** tudo sai do registro de tarefas e dos modelos Pydantic, com o mesmo texto que alimenta o editor de parâmetros. Um teste falha se algum parâmetro estiver sem descrição.
- **Mapa de migração:** a tabela lista também tarefas do IRAF ainda ausentes, com status *disponível*, *planejada*, *substituída* ou *não será portada*. Os 609 nomes do inventário são a base. Assim `harwood help rfits` responde "desnecessária: o FITS é lido diretamente" em vez de "comando não encontrado".
- **Pacotes como presets:** o IRAF repete `apall` em oito pacotes mudando só os padrões. No harwood a tarefa aparece uma vez e a tabela mostra os presets (`echelle`, `hydra`, `kpnoslit`…).
- **Resposta instantânea:** `list` e `--help` não podem importar astropy nem photutils. Módulos de parâmetros leves; imports pesados só dentro da execução. O Click tem padrão documentado para carregar subcomandos sob demanda.
- **`?` e `??` só dentro da sessão:** no bash e no zsh, `?` é curinga. Na linha de comando ficam as palavras; na sessão do harwood funcionam como no IRAF.

- **Autocompletar e `--json`:** o Typer já oferece autocompletar no shell; estendido a nomes e valores de parâmetros, a descoberta fica quase automática. A saída em JSON serve a scripts e ao editor de parâmetros.

4. Distribuição e instalação

No esquema do IRAF, mas leve. O peso real vem da pilha científica, não do harwood: leve significa poucas dependências obrigatórias, extras opcionais e nenhuma configuração de sistema — é aí que o IRAF dói.

Aspecto	Decisão
Instalar	Um comando, sem compilar nada: `uv tool install harwood` ou `pipx install harwood`; o uv baixa a versão certa do Python
Sessão	`harwood` sem argumentos abre uma sessão com as tarefas carregadas — o equivalente ao `cl>`. As mesmas tarefas rodam direto do bash
Por que a sessão	Importar astropy, scipy e matplotlib pode passar de um segundo; pagar isso a cada comando irrita. O IRAF resolveu o mesmo problema mantendo executáveis carregados (`prcache`)
Começar uma redução	`harwood init` no diretório, no lugar do `mkiraf`: cria arquivos de projeto visíveis e comentados
Dados de calibração	Estrelas padrão, extinção e listas de linhas num pacote de dados versionado, instalado junto. **Verificar licenças antes de redistribuir os do IRAF**
Diagnóstico	`harwood doctor` confere o ambiente (Python, DS9, X11 ou Wayland)
Extras opcionais	`harwood[spec]`, `harwood[ds9]`, `harwood[gui]`; um `harwood-core` sem interface gráfica para servidores

5. Editor de parâmetros

Mais fácil que o do IRAF e o do MESA, e explicando o que cada coisa faz. Como os parâmetros já são modelos Pydantic, o editor é gerado a partir deles.

Recurso	Descrição
Explicação por parâmetro	Escrita no próprio modelo: o que faz, unidade, quando mudar, valores típicos, referência. O mesmo texto alimenta editor, `help` e documentação — nunca dessincronizam
Parâmetros irrelevantes apagados	Conforme o contexto: `nlow` e `nhigh` só acendem com `--reject minmax`
Origem de cada valor	Visível, com destaque para o que difere do padrão
Validação ao editar	Em vez de descobrir o erro ao rodar
Plano de execução	Painel que se atualiza conforme os valores mudam (seção 2.1)

Interface proposta: TUI com a biblioteca Textual — rica no terminal, com mouse e painel de ajuda, leve, offline e funcional por SSH, o que importa quando a redução roda num servidor do observatório. Uma versão em Qt vem depois, a partir do mesmo esquema, integrada à janela do visualizador (seção 8).

6. Idioma

Padrão: inglês. Margaret Harwood era estadunidense e o inglês é a língua franca científica. **Português e espanhol** também disponíveis — um diferencial raro, já que ferramentas de redução quase nunca são traduzidas, e uma resposta direta à questão dos bastidores: explicar parâmetros na língua dos estudantes.

Decisão	Detalhe
Pacote único, não downloads separados	Idioma vem do `LANG`, de configuração do usuário ou de `--lang`. Versões separadas obrigariam a lançar cada correção três vezes e acabariam se desencontrando. Traduzidas ficam a página de download e o guia de instalação
Escopo da tradução	`--help` e a explicação de cada parâmetro

Decisão	Detalhe
Fica em inglês	Mensagens de erro, logs, plano de execução e proveniência — quem cola um erro num fórum encontra resposta porque todos veem o mesmo texto
Nunca traduzidos	Nomes de tarefas, parâmetros e valores; siglas (FWHM, PSF, WCS); nomes de algoritmos (zscale, DAOPHOT); jargão que a comunidade usa em inglês (bias, flat, dark, overscan, fringe, seeing, crosstalk)
Glossário no repositório	Fixa os termos e evita que cada tradutor decida sozinho. Traduzir jargão pode mudar o sentido: "bias" como "viés" ou "sesgo" vira viés estatístico
Casos de fronteira	Termos com tradução consolidada em cursos e artigos ("largura equivalente", "velocidade radial", "massa de ar") — decidir no glossário se entram como exceção

O texto traduzido apresenta o termo sem traduzi-lo: "*Subtrai o bias (nível de base introduzido pela eletrônica do detector)…*".

6.1 Mecânica

Com escopo restrito a `--help` e parâmetros, arquivos por tarefa e idioma, organizados por nome de parâmetro, funcionam melhor que o gettext. A chave é o próprio identificador:

```
# locale/pt_BR/imcombine.yaml
summary: Combina imagens pixel a pixel
params:
  reject:
    source: 5c1e9a02    # hash do texto em inglês que foi traduzido
    short: Algoritmo para descartar pixels desviantes antes de combinar
    long: |
      Com `ccdclip`, o limite de rejeição vem do modelo de ruído do CCD
      (`gain`, `rdnoise`, `snoise`), e não do espalhamento da pilha,
      o que ajuda quando há poucas imagens para combinar.
```

Três verificações automáticas:

- **Tradução desatualizada:** se o texto em inglês mudou, o hash não bate; o `--help` mostra o inglês com aviso de tradução pendente — informação correta vale mais que idioma.
- **Identificadores intactos:** nomes entre crases precisam ser idênticos ao original, para ninguém traduzir `ccdclip` sem querer.
- **Glossário respeitado:** teste procura traduções vetadas (como "viés") e acusa.

Escrever o inglês pensando em quem traduz: frases curtas, sem expressões idiomáticas, identificadores sempre entre crases. Para o espanhol, mirar um espanhol neutro latino-americano, com revisão de alguém da comunidade astronômica chilena, argentina ou mexicana.

6.2 Armadilhas técnicas

- **Cabeçalhos FITS só aceitam ASCII:** cartões `HISTORY` e keywords de proveniência não comportam "ção" nem "ñ" — ficam sempre em inglês. Relatórios para leitura podem ser traduzidos; o que é gravado nos dados, não.
- **Números nunca são localizados:** ponto decimal em qualquer idioma. O harwood não deve alterar o `LC_NUMERIC` do processo, senão bibliotecas em C que leem números em texto passam a esperar vírgula.
- **Tradução na hora de exibir:** descrições ficam em inglês nos modelos e são traduzidas na exibição, não na importação do módulo — caso contrário o idioma congela no primeiro carregamento.

7. Onde o Python não basta

O Python tropeça em laços pixel a pixel com lógica condicional: rejeição iterativa no combine, raios cósmicos, extração ótima com limpeza. Como o harwood orquestra em vez de reimplementar, muito cálculo pesado já é compilado dentro de astropy, scipy e photutils; a segunda linguagem só entra nos algoritmos que o harwood implementar sozinho.

Decidido: Numba primeiro; Rust (PyO3 + maturin) para núcleos estáveis e críticos. Descartadas: Julia (runtime embutido e latência na primeira chamada brigam com a instalação simples), C++ (traz de volta a complexidade que se quer evitar) e Cython (legítimo, mas no fundo é C com cara de Python).

7.1 Onde as linguagens entram

Numba e Rust aparecem numa camada só: funções que recebem arrays NumPy e números e devolvem arrays. O resto é Python.

```
tarefa (modelo Pydantic, FITS, proveniência, interface)      ← Python
        │  só arrays NumPy e números atravessam esta linha
núcleo  sigclip_mean(stack, lsigma, hsigma, maxiter)
        ├── reference   NumPy puro: a especificação, legível
        ├── numba       o mesmo algoritmo em laço, compilado na hora
        └── rust        módulo compilado, importado como módulo Python
```

Um despachante escolhe o backend: o mais rápido instalado, ou o pedido em `HARWOOD_BACKEND`. Isso permite rodar uma redução inteira em modo referência quando algo parecer estranho. **O backend usado e sua versão vão para o log de proveniência.**

Regra que amarra tudo à transparência: código compilado só em núcleos numéricos, e sempre com implementação de referência em Python legível, testada contra a versão rápida. Quem quiser entender o que acontece por trás dos panos lê o Python; a versão compilada é otimização verificada.

7.2 Numba

Dependência comum, sem etapa de compilação nem arquivo novo: o laço é Python com um decorador.

Aspecto	Detalhe
Compilação	Na primeira chamada do processo. Com `cache=True`, as seguintes só carregam o cache. O cache deve ficar em `~/.cache/harwood`, via `NUMBA_CACHE_DIR`
Limitação	Só enxerga arrays e números — modelos Pydantic, objetos da astropy e dicionários ficam de fora. Mais um motivo para a camada de núcleos ser pura
`fastmath=True`	Evitar: permite reordenar operações e ignorar NaN, o que muda resultados e quebra a reprodutibilidade
Compatibilidade	Numba 0.67 já suporta Python 3.14

7.3 Rust

Código compilado no repositório: `Cargo.toml` e `src/lib.rs`, com maturin como backend de build no `pyproject.toml`. A função aparece para o Python como qualquer outra.

Aspecto	Detalhe
Desenvolvimento	`maturin develop --release` compila e instala em modo editável
Distribuição	Wheel `abi3`: um arquivo (~390 KB no protótipo) serve para qualquer Python ≥ 3.10, sem recompilar a cada versão nova. Import em ~1 ms. **O usuário final não precisa ter Rust**
Compilar em CI	Em container manylinux antigo; o wheel do protótipo saiu marcado para glibc 2.34 (Ubuntu 24.04) e não rodaria em distribuições mais velhas
Organização	Núcleos Rust num pacote separado no mesmo repositório (`harwood-kernels`): quem contribui só em Python não instala Rust, e o despachante recorre a Numba ou à referência quando o módulo compilado falta
Paralelismo	`rayon` com o GIL solto (`py.detach`)

7.4 Prova de conceito e medições

Núcleo implementado três vezes: média com sigma clipping pixel a pixel, versão simplificada do `imcombine --reject sigclip`. Pilha 20 × 1024 × 1024, sandbox com 1 CPU:

Backend	Tempo	Observação
reference	5,2 s	NumPy puro
numba	0,6 s	compilação sem cache ~4,4 s; carga do cache ~0,15 s
rust	0,5 s	import ~1 ms; wheel ~390 KB

Com mais núcleos, Numba (`prange`) e Rust (`rayon`) escalam; a referência não. Com o empate de velocidade, a escolha vira engenharia: **Numba** atende quase tudo sem mudar o fluxo de trabalho; **Rust** vale para núcleos estáveis e muito usados, onde a compilação num processo sem cache incomodaria na linha de comando, ou onde a lógica excede o subconjunto de Python que o Numba aceita.

7.5 O que os testes exigem

- **Contagens de rejeição idênticas** entre backends, não só valores próximos: decisões discretas (rejeitar ou não um pixel) podem mudar sem que a média ultrapasse a tolerância numérica.
- **Limitação conhecida documentada como teste:** dois raios cósmicos no mesmo pixel inflam o sigma da pilha e escapam do sigclip clássico — com 15 imagens e ruído de 10 ADU, o sigma sobe a ~1900 e o limite de 3σ passa de 5000. É a razão de o IRAF oferecer `ccdclip` e `crreject`, que tiram o sigma do modelo de ruído do CCD.

8. Visualizador nativo

Decidido: visualizador próprio, numa janela só; DS9 apenas como último recurso.

8.1 Pilha escolhida

Componente	Motivo
PySide6 (Qt 6)	Licença LGPL — o PyQt6 é só GPL e forçaria o harwood inteiro a ser GPL. Basta `PySide6-Essentials` (~80 MB); os 175 MB de Addons não são necessários. Roda nativo em X11 e Wayland, o que elimina o problema do DS9 via XWayland
pyqtgraph	MIT, feito para imagens e gráficos científicos interativos, rápido em arrays grandes
astropy	zscale, stretches e WCS. O visualizador não precisa do matplotlib, que fica só para figuras de publicação
regions	Lê e escreve arquivos de região do DS9 — troca de regiões sem depender dele

Construir em vez de embutir o Ginga (visualizador em Python no estilo DS9, maduro, BSD, embutível em Qt): o diferencial do harwood é a integração — teclas que chamam tarefas, sobreposições vindas das tarefas (aberturas, marcas do `tvmark`, máscaras, aberturas do `apedit`), coordenadas 1-based e interações registradas na proveniência. Com um visualizador genérico, cada uma dessas exigiria camada de adaptação. O Ginga fica como plano B.

8.2 Arquitetura

Tarefas nunca importam Qt: falam com uma interface de display — o equivalente ao `stdimage` do IRAF — com três implementações:

Backend	Uso
`native`	A janela do harwood; padrão
`headless`	Salva PNG; SSH, lote e testes automáticos
`ds9`	Via XPA ou SAMP; último recurso

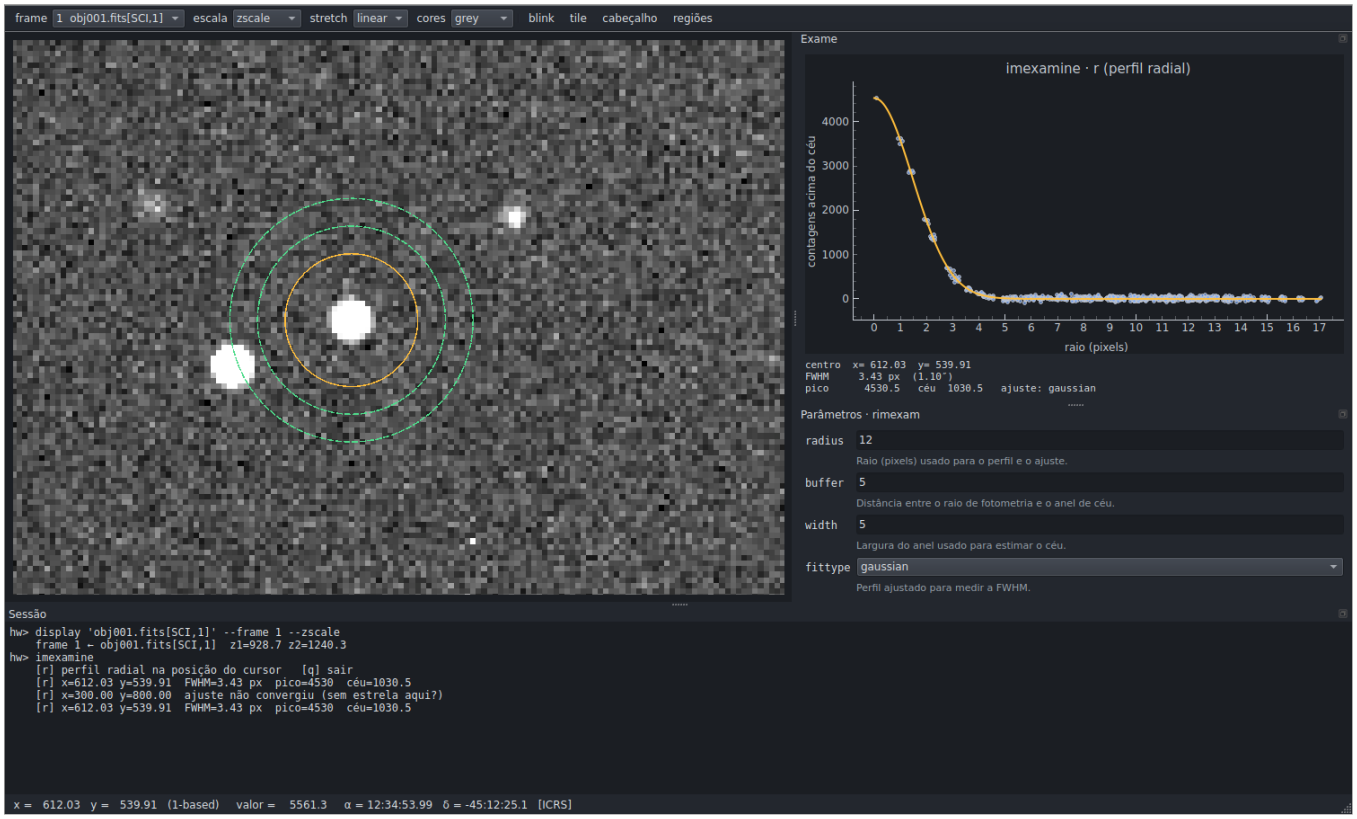
Mesmo princípio dos núcleos numéricos: interface estável, implementações trocáveis. Daí dois modos de uso: **`harwood gui`**, com o ambiente inteiro na janela (o `qtconsole` permite embutir um console IPython no painel de sessão, e o editor de parâmetros vira a versão Qt do mesmo esquema Pydan-

tic); e **sessão no terminal**, onde `display` abre ou reaproveita uma única janela e o prompt continua respondendo, porque o IPython sabe rodar o laço de eventos do Qt. A TUI continua sendo o caminho por SSH.

8.3 Protótipo

Cerca de 250 linhas, em uma janela só: imagem com zscale, tecla `r` do `imexamine` com perfil radial e ajuste, círculos de abertura e céu sobre a estrela, painel de parâmetros com explicação, log da sessão e barra de status com coordenadas 1-based e RA/Dec. Os painéis são docks: podem ser arrastados, empilhados em abas ou soltos como janela flutuante.

Validação: estrela sintética injetada em (612,0; 540,0) na convenção IRAF; o protótipo mediu (612,03; 539,91), com FWHM de 3,43 px contra 3,40 injetados — confirma que a conversão de meio pixel está correta.



Captura real do protótipo: imagem com zscale, perfil radial da tecla `r` com ajuste gaussiano, círculos de abertura e céu, painel de parâmetros com explicação, log da sessão e barra de status com coordenadas 1-based e RA/Dec.

8.4 Cuidados

Ponto	Detalhe
Meio pixel	No pyqtgraph o pixel *i* ocupa [*i*, *i*+1); no IRAF o centro do pixel 1 fica em 1,0. Logo `x_iraf = x_cena + 0,5`. Erro aqui desloca todas as medidas — o teste com estrela sintética deve ser permanente
Imagens grandes	A tela mostra versão normalizada e o zscale usa amostra de pixels; as medidas usam sempre os dados originais em ponto flutuante. Para CCDs de 4k, gerar versões reduzidas
Teclado	Minúsculas pertencem à tarefa; maiúsculas e comandos `:` pertencem à janela — como no modo cursor do IRAF
Uso remoto	Encaminhamento X11 funciona mas é lento, e o Wayland não tem transparência de rede. No servidor, usar `headless` ou rodar localmente
Peso	O Qt vira a maior dependência — daí o `harwood-core` sem interface gráfica

8.5 Escopo do visualizador

MVP (substitui o uso diário do DS9 com o IRAF): frames com blink, zscale e minmax, stretches, mapas de cores, zoom e pan, leitura de coordenadas e WCS, seleção de extensão MEF, visor de cabeçalho, teclas do `imexamine`, marcas do `tvmark`.

Depois: regiões DS9, contornos, cubos, grade de coordenadas (`wcslab`), edição de máscaras (`bpmedit`), aberturas sobre espectros 2D (`apedit`).

9. Estado atual do código

Item	Situação
Arquitetura central	Modelos Pydantic que geram simultaneamente CLI, validação, JSON Schema e proveniência; registro por decorador `@register`
Tarefas	`combine`, `ccdproc`, `daofind`, `apphot`; sistema de receitas YAML
Testes	Suíte de 23 testes, com regressões para os três bugs encontrados (cartões `IMCMB` ausentes, condição de corrida na proveniência, coluna silenciosamente trocada após a mudança de API do photutils 3.0)
Teste sintético	95% de completeza em dados de verdade conhecida
Camada interativa	matplotlib com eventos de teclado e mouse — **a rever**, dada a decisão da seção 8 (PySide6 + pyqtgraph). A abstração `CursorBackend` planejada continua válida

9.1 Pendências que as decisões acima criaram

- **`epar` grava em `$HARWOOD_HOME/params`** — isso é o `uparm$` de novo. Precisa migrar para o diretório da redução (seção 2).
- **Tarefa chamada `apphot`** — no IRAF esse é o nome do *pacote*; a tarefa é `phot` (seção 3).
- **Tarefa `combine`** — decidir o que espelha: o `imcombine` genérico ou o `combine` do `ccdred`, que entende tipos de imagem e filtros.
- **Camada interativa em matplotlib** — substituir pela pilha Qt.

10. Em aberto

1. Definir o escopo do MVP percorrendo os 22 domínios da seção 3 do inventário e marcando cada um como **núcleo**, **depois** ou **fora de escopo**. A espectroscopia é a maior lacuna do ecossistema Python e o maior valor potencial do harwood, especialmente para astrofísica estelar.
2. Resolver as quatro pendências da seção 9.1.
3. Verificar as licenças dos dados de calibração do IRAF antes de redistribuí-los.
4. Fechar o glossário de tradução, incluindo os casos de fronteira.
5. Decidir a convenção de extensões para variância e máscara (`SCI`/`VAR`/`DQ` ou outra).
6. Definir a política de compatibilidade: o que garante estabilidade de nomes de tarefas e parâmetros entre versões.